



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS ARARANGUÁ
ENGENHARIA DE COMPUTAÇÃO

Emanuela Kemper Schmitz e Luan Gabriel

Relatório Técnico: Trabalho 04

Araranguá

2026



Emanuela Kemper Schmitz e Luan Gabriel

Relatório Técnico: Trabalho 04

Proposta apresentada como requisito parcial para aprovação na disciplina de DEC7555 Linguagem e Descrição de *Hardware*, do curso de Engenharia da Computação da Universidade Federal de Santa Catarina. Prof. Dr. Ney Laert Vilar Calazans.

Araranguá

2026

RESUMO

Este relatório detalha o desenvolvimento e a validação em duas etapas de um algoritmo de transferência massiva de dados para a arquitetura MIPS_S. O Projeto 2 foca na emulação em software via MARS, onde uma rotina em Assembly MIPS foi desenvolvida para transferir caracteres entre blocos de memória, valendo-se temporariamente de chamadas de sistema para a carga inicial de um arquivo de texto. O Projeto 3 consolida a transição para o nível físico, validando o mesmo algoritmo diretamente no simulador de hardware MIPS_S_Sim. O processamento destes arquivos gerou o módulo `strcpy_large-strings.txt`, injetando de forma estática o código e os dados nas BRAMs do FPGA. Os resultados da simulação em hardware atestam a robustez e a exatidão do laço de cópia, homologando a infraestrutura de memória necessária para a futura integração de módulos periféricos no Projeto 4

Palavras-chave: VHDL; FPGA; Máquina de Estados Finita (FSM);.

LISTA DE FIGURAS

Data Segment 0x10014000.....	8
Data Segment 0x10010000.....	8
Simulação do sistema.....	10

SUMÁRIO

1 INTRODUÇÃO.....	5
2 Transferência de Memória em Assembly MIPS (Projeto 2).....	6
2.1 Estruturação do Mapa de Memória.....	6
2.2 Carga do Arquivo em Memória.....	7
2.3 A lógica da Cópia (rotina strcpy).....	7
2.4 Simulação e Resultados no MARS.....	7
Data Segment 0x10014000.....	8
Data Segment 0x10010000.....	8
3 PROJETO 3 — SIMULAÇÃO DO SOFTWARE SOBRE O HARDWARE DO PROCESSADOR.....	9
3.1 Adaptação do Programa (strcpyMARS.asm).....	9
3.2 Geração e Preparação dos Dumps de Memória.....	9
3.3 Simulação do Sistema MIPS_S_withBRAMs.....	10
CONCLUSÃO.....	10

1 INTRODUÇÃO

A integração entre rotinas de software de baixo nível e arquiteturas de hardware customizadas é uma etapa crítica no fluxo de desenvolvimento de sistemas digitais. Este relatório detalha a implementação e a validação técnica dos Projetos 2 e 3, fases integrantes de uma abordagem incremental cujo objetivo geral é lidar com um sistema computacional complexo (MIPS_S_withBRAMs) e desenvolver o hardware de um periférico capaz de interagir com ele.

A estruturação lógica inicia-se no Projeto 2, que foca no desenvolvimento de um programa robusto em linguagem de montagem (Assembly MIPS) para a transferência em massa de dados textuais, garantindo o rigoroso alinhamento do mapa de memória através do simulador MARS. Na sequência, o Projeto 3 eleva o critério de validação ao transpor essa rotina de cópia diretamente para o ambiente de simulação em hardware, isolando as dependências de chamadas de sistema do hospedeiro. Em conjunto, essas etapas homologam tanto a eficiência do algoritmo quanto a arquitetura física de memória, consolidando a infraestrutura de dados que suportará a implementação final na arquitetura FPGA.

2 Transferência de Memória em Assembly MIPS (Projeto 2)

O objetivo central foi a criação de um software em Assembly para a arquitetura MIPS, capaz de realizar a transferência massiva de dados (texto) entre diferentes regiões de memória. O projeto exigiu a adequação o mapa de memória do sistema MIPS_S_withBRAMs, garantindo que o buffer de origem fosse alocado no endereço 0x10014000 e o destino no endereço 0x10010000. Para a validação no simulador MARS, implementou-se uma sub-rotina de entrada e saída que utiliza chamadas de sistema (syscalls) para leitura de um arquivo externo contendo um trecho da obra Romeu e Julieta de William Shakespeare. Posteriormente, desenvolveu-se a sub-rotina 'strcpy' para executar a cópia byte a byte do texto, encerrando a operação ao detectar o caractere nulo (0x00). A execução do código atendeu a todas as restrições da especificação, preparando com sucesso a estrutura de dados (Data Segment) que será convertida para uso nas BRAMs da FPGA nos Projetos 3 e 4.

O desenvolvimento do código foi dividido em três frentes principais: a estruturação das diretivas de dados (.data), a sub-rotina de leitura do arquivo no ambiente hospedeiro e a sub-rotina de cópia de strings em si.

2.1 ESTRUTURAÇÃO DO MAPA DE MEMÓRIA

Esta estruturação foi fornecida pelo professor, conforme o ambiente MARS inicializa o segmento de dados no endereço base 0x10010000. Para satisfazer os requisitos espaciais definidos na especificação, foram aplicadas reservas de espaço (.space):

- Destino (MIPS_S): O rótulo 'dest_text_addr' foi alocado logo no início (0x10010000). A partir dele, reservou-se um bloco quase equivalente a 9 Kbytes (.space 9200).
- Alinhamento do Periférico: Para garantir que o endereço do texto fonte (source_text_addr) caísse exatamente na fronteira 0x10014000, utilizou-se um bloco de preenchimento (padding) de 7144 bytes.
- Fonte (Periférico): O rótulo 'source_text_addr' marca o início do texto e conta com mais 9200 bytes reservados para o armazenamento do arquivo de leitura.

2.2 CARGA DO ARQUIVO EM MEMÓRIA

Como o processador MIPS_S puramente em hardware não possui sistema operacional nativo para leitura de arquivos textuais, o Projeto 2 requer o uso do simulador

MARS para emular essa funcionalidade [cite: 5_4-MIPS_S_withBRAMS_Presentation_V5.9.pdf]. Isso foi solucionado através da sub-rotina 'carrega_arquivo', que realiza o salvamento do contexto de retorno (\$ra) na pilha (\$sp) e faz uso de System Calls:

- Syscall 13 (Open): Abre o arquivo 'RomeoJuliet_Act2-Scene2.txt' referenciado pelo diretório absoluto salvo em .asciiz no modo 'Somente Leitura'. O descritor de arquivo gerado é salvo no registrador de salvamento (\$s0).
- Syscall 14 (Read): Com o descritor validado, o MARS despeja todo o conteúdo do arquivo diretamente no endereço físico apontado por 'source_text_addr' (0x10014000).
- Syscall 16 (Close): Executa o fechamento do arquivo lido para liberação de memória.

2.3 A LÓGICA DA CÓPIA (ROTINA STRCPY)

A etapa principal do sistema ocorre no laço 'strcpy'. A sub-rotina recebe os endereços de fonte (0x10014000) no registrador \$a0 e de destino (0x10010000) em \$a1. O algoritmo opera varrendo caractere por caractere. Uma instrução Load Byte (lb) extrai o caractere atual da fonte para o registrador temporário \$t0, e uma instrução Store Byte (sb) imediatamente o grava no endereço de destino. A verificação de término ocorre através da instrução branch-on-equal (beq), que avalia se o caractere transferido é igual a nulo (\$zero).

Caso os dados terminem com um caractere NULL (0x00), o laço quebra e a função retorna o controle à main. Caso contrário, a aritmética de ponteiros avança os registradores \$a0 e \$a1 em 1 byte, mantendo o processo sequencial.

2.4 SIMULAÇÃO E RESULTADOS NO MARS

A compilação e validação do programa foram executadas no simulador MARS. O fluxo de validação focou na análise da aba 'Data Segment' do simulador.

1. Configuração e Leitura: Ao iniciar a execução (run), atestou-se o sucesso das chamadas de sistema, com a transferência instantânea dos caracteres ASCII do script de Shakespeare para a faixa de memória do periférico iniciada em 0x10014000.

2. Execução do Strcpy: A rotina provou sua robustez iterando sobre os milhares de caracteres em frações de segundo. Ao final da execução, inspecionou-se a base da memória de dados primária (0x10010000). Comprovou-se que a totalidade do texto havia sido fielmente copiada para o destino, sem vazamentos de memória ou estouros de buffer.

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10014000	g a r T	y d e	R f o	o e m o	d n a	l u J	\r t e i	A \n \r \n
0x10014020	2 t c	c S ,	e n e	\r \n \r 2	a C " \n	e l u p	s ' t	h c r o
0x10014040	. d r a	\r \n \r "	n E " \n	r e t	E M O R	\n \r " O	O R \n \r	\r O E M
0x10014060	e H \n	t s e j	t a s	a c s	t s r	t a h	e v e n	e f r
0x10014080	a t l	u o w	\r . d n	" \n \r \n	I L U J	a T E	a e p p	a s r
0x100140a0	e v o b	t a	i w a	w o d n	\r \n \r "	t u B \n	o s ,	! t f
0x100140c0	t a h w	g i l	t t h	u o r h	y h g	e d n o	i w r	w o d n
0x100140e0	e r b	? s k a	t I \n \r	s i	e h t	t s a e	n a ,	u J d
0x10014100	t e i l	s i	e h t	. n u s	r A \n \r	, e s i	i a f	u s r
0x10014120	a , n	k d n	l l i	e h t	i v n e	s u o	n o o m	W \n \r ,
0x10014140	i o h	l a s	d a e r	i s y	a k c	p d n	e l a	h t i w
0x10014160	i r g	\r , f e	a h T \n	h t t	h u o	m r e	d i a	t r a
0x10014180	r a f	e r o m	i a f	h t r	s n a	\r : e h	e B \n	t o n
0x100141a0	r e h	d i a m	i s ,	e c n	e h s	e s i	o i v n	\r ; s u
0x100141c0	r e H \n	s e v	l a t	e v i l	i y r	u b s	i s t	a k c
0x100141e0	g d n	n e e r	n A \n \r	o n d	b e n	f t u	s l o o	o d

Data Segment 0x10014000

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	e h T	g a r T	y d e	R f o	o e m o	d n a	l u J	\r t e i
0x10010020	A \n \r \n	2 t c	c S ,	e n e	\r \n \r 2	a C " \n	e l u p	s ' t
0x10010040	h c r o	. d r a	\r \n \r "	n E " \n	r e t	E M O R	\n \r " O	O R \n \r
0x10010060	\r O E M	e H \n	t s e j	t a s	a c s	t s r	t a h	e v e n
0x10010080	e f r	a t l	u o w	\r . d n	" \n \r \n	I L U J	a T E	a e p p
0x100100a0	a s r	e v o b	t a	i w a	w o d n	\r \n \r "	t u B \n	o s ,
0x100100c0	! t f	t a h w	g i l	t t h	u o r h	y h g	e d n o	i w r
0x100100e0	w o d n	e r b	? s k a	t I \n \r	s i	e h t	t s a e	n a ,
0x10010100	u J d	t e i l	s i	e h t	. n u s	r A \n \r	, e s i	i a f
0x10010120	u s r	a , n	k d n	l l i	e h t	i v n e	s u o	n o o m
0x10010140	W \n \r ,	i o h	l a s	d a e r	i s y	a k c	p d n	e l a
0x10010160	h t i w	i r g	\r , f e	a h T \n	h t t	h u o	m r e	d i a
0x10010180	t r a	r a f	e r o m	i a f	h t r	s n a	\r : e h	e B \n
0x100101a0	t o n	r e h	d i a m	i s ,	e c n	e h s	e s i	o i v n
0x100101c0	\r ; s u	r e H \n	s e v	l a t	e v i l	i y r	u b s	i s t
0x100101e0	a k c	g d n	n e e r	n A \n \r	o n d	b e n	f t u	s l o o

Data Segment 0x10010000

Como passo adjacente à conclusão desta etapa (que abre caminho para o Projeto 3), foi realizado o procedimento de Dump Memory To File. Exportou-se o código objeto (prog.txt) da janela de texto, e as áreas de dados (data.txt) com o texto já copiado, que servirão como massa de dados para inicializar as BRAMs do processador MIPS em VHDL.

Como passo adjacente à conclusão desta etapa (que abre caminho para o Projeto 3), foi realizado o procedimento de Dump Memory To File. Exportou-se o código objeto (prog.txt) da janela de texto, e as áreas de dados (data.txt) com o texto já copiado, que servirão como massa de dados para inicializar as BRAMs do processador MIPS em VHDL.

3 SIMULAÇÃO DO SOFTWARE SOBRE O HARDWARE DO PROCESSADOR (PROJETO 3)

O Projeto 3 teve como escopo principal validar a execução do algoritmo de transferência de dados estruturado no Projeto 2 diretamente sobre o hardware do processador

MIPS_S. Esta etapa é estritamente necessária para atestar a confiabilidade do código fora do ambiente idealizado do simulador MARS, além de consolidar a infraestrutura de memória física e os respectivos *dumps* que comporão o sistema integrado no Projeto 4.

3.1 ADAPTAÇÃO DO PROGRAMA (STRCPYMARS.ASM)

Dado que a arquitetura em hardware do processador MIPS_S não dispõe de um sistema operacional para suporte a chamadas de sistema (tais como abertura e leitura de arquivos via *syscalls*), o código-fonte original precisou ser refatorado. O programa *strcpyMARS_PJ_03.asm* foi derivado do *strcpyMARS.asm* mediante a supressão das rotinas de manipulação de arquivo. Como premissa, assumiu-se que o texto já estaria fisicamente pré-carregado na área de origem da memória (endereço **0x10014000**) através da inicialização das BRAMs. Dessa forma, o fluxo de execução do software foi reduzido exclusivamente ao laço de cópia para o destino (**0x10010000**). Adicionalmente, implementou-se um laço infinito ao término das instruções para estabilizar o estado do processador, o que provê um marcador visual claro da conclusão da rotina durante a análise das formas de onda no simulador.

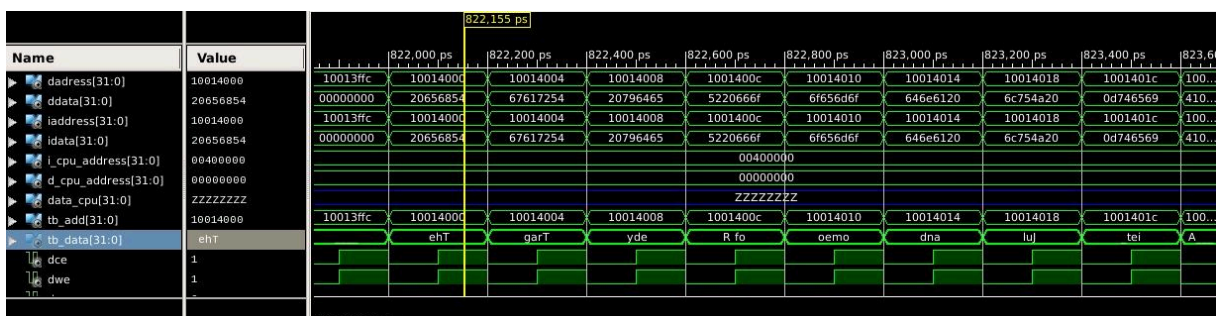
3.2 GERAÇÃO E PREPARAÇÃO DOS DUMPS DE MEMÓRIA

A extração dos *dumps* de memória exigiu uma abordagem fracionada devido a limitações (bugs) documentadas do simulador MARS (versão 4.5) ao processar grandes blocos de endereçamento, como os 32 Kbytes alocados para este escopo. O procedimento adotado seguiu três etapas fundamentais: primeiro, extraiu-se o *dump* do segmento de texto (**DumpTextPJ3.txt**) a partir do código refatorado para hardware; em seguida, executou-se o código original do Projeto 2 no MARS para forçar o carregamento do texto em memória e gerar um *dump* de dados (**DumpDataPJ2.txt**) contendo o arquivo de origem. Para reestabelecer as condições iniciais de simulação, o arquivo **DumpDataPJ2.txt** foi editado manualmente, sobrescrevendo a área de destino com zeros e eliminando os dados residuais da execução prévia. Finalmente, o script **strcpy_large-strs.txt** contendo ambos os dumps de memória foi manualmente gerado.

3.3 SIMULAÇÃO DO SISTEMA MIPS_S_WITHBRAMs

A validação funcional do Projeto 3 foi conduzida no ambiente **MIPS_S_Sim**, valendo-se do *testbench* **MIPS_S_Sim_tb.vhd**. A adoção deste *testbench* específico otimizou

o fluxo de validação ao realizar a leitura direta dos arquivos texto ([DumpTextPJ3.txt](#) e [DumpDataPJ2.txt](#)) durante o ciclo de *reset*, eliminando a necessidade de instanciar BRAMs estáticas apenas para a etapa de inicialização. A análise temporal confirmou que o programa [strcpyMARS_PJ_03.asm](#) itera perfeitamente sobre o hardware do MIPS_S, replicando o comportamento validado no simulador de software e finalizando a transferência de bytes de forma exata ao encontrar o caractere nulo.



Simulação do sistema.

CONCLUSÃO

A execução sequencial e interdependente dos Projetos 2 e 3 atestou com sucesso a viabilidade de transpor um algoritmo concebido em um emulador de software (MARS) para a realidade física do hardware processador (MIPS_S). A sub-rotina de cópia iterativa em Assembly provou sua estabilidade operando perfeitamente dentro das rígidas fronteiras de endereço predefinidas, manipulando milhares de bytes até a detecção do caractere de parada. Ademais, o procedimento metodológico de tratamento e intervenção nos dumps de memória contornou com precisão as anomalias de exportação do simulador, garantindo uma inicialização limpa e exata das BRAMs.

Com a validação funcional do software sobre o hardware do MIPS_S concluída, o projeto consolida a infraestrutura de dados necessária para avançar em direção à síntese física. O sistema encontra-se estruturalmente maduro para a fase final, que consiste em criar um módulo periférico dedicado contendo uma máquina de estados e memória local, projetado para se acoplar à interface do MIPS_S_withBRAMs e permitir que o usuário externo gerencie de forma autônoma a transferência e a visualização dos dados via hardware de entrada e saída.

